Báo Cáo Nghiên Cứu Và Lộ Trình Triển Khai Ứng Dụng RAG PDF-Chatbot Tiêu Chuẩn Doanh Nghiệp 2026 (Mô Hình Nhóm 4 Người)

1. Mục Tiêu Dự Án Và 3 Giá Trị Cốt Lõi Đạt Được

Dự án "Advanced PDF-Chatbot 2026-Ready" không chỉ là một bài tập thực hành mà là một hệ thống phần mềm có tính ứng dụng thực tiễn cao, được mô phỏng chính xác theo quy trình MLOps của các tập đoàn công nghệ. Hoàn thành dự án này, nhóm sẽ đạt được 3 giá trị cốt lõi:

1. **Sở hữu một Product cấp doanh nghiệp:** Một hệ thống AI đa tầng, có kiến trúc phân tán (Microservices), xử lý hàng ngàn tài liệu với độ trễ thấp và bảo mật cao, thoát khỏi định dạng kịch bản Jupyter tĩnh.
2. **Portfolio sắc bén cho CV & LinkedIn:** Chứng minh năng lực thực chiến trọn vẹn vòng đời phát triển phần mềm AI (từ data ingestion, AI orchestration, backend, frontend đến CI/CD). Đây là "vũ khí" thu hút nhà tuyển dụng mạnh mẽ nhất.
3. **Thành thạo bộ kỹ năng công cụ công nghiệp:** Cả 4 thành viên đều sẽ nắm vững và sử dụng thực tế hệ sinh thái làm việc nhóm chuyên nghiệp, bao gồm GitFlow trên GitHub, container hóa bằng Docker, và tự động hóa kiểm thử/đánh giá mô hình bằng Postman.

2. Cấu Trúc Đội Ngũ 4 Người & Quy Tắc Vận Hành Đa Chéo

Dựa trên kiến trúc chuẩn của một dự án phần mềm AI hiện đại, nhóm 4 người sẽ được chia thành 2 nhánh chuyên môn: Nhánh AI (2 người) và Nhánh Web/Software (2 người). Tuy nhiên, **nguyên tắc bắt buộc** là cả 4 người đều phải thao tác thuần thục GitHub, Docker và Postman để không tạo ra điểm nghẽn (bottleneck) trong quy trình CI/CD.

- **Thành viên 1 (AI - Data & ML Engineer):** Đam mê dữ liệu và thuật toán nhúng. Phụ trách pipeline đưa PDF thô vào Vector Database (Ingestion), chiến lược cắt đoạn (Chunking) theo bố cục, và quản trị Vector Database.
- **Thành viên 2 (AI - AI Orchestrator):** Đam mê tư duy logic và kiến trúc mô hình. Phụ trách xây dựng tư duy đa bước bằng LangChain, tích hợp LLM mã nguồn mở từ Hugging Face, và lập trình các lớp rào chắn (Guardrails) đánh giá độ tự tin (Confidence Score > 0.7).
- **Thành viên 3 (Web - Backend & DevOps Engineer):** Đam mê hiệu suất và hệ thống mạng. Xây dựng lõi API bằng FastAPI, thiết lập vòng lặp thử lại API (Retry/Backoff) chống sụp đổ, và phụ trách chính kiến trúc Docker Compose cho toàn bộ nhóm.

- **Thành viên 4 (Web - Frontend & QA Engineer):** Đam mê trải nghiệm người dùng (UX) và quản trị chất lượng. Phát triển giao diện Streamlit truyền phát dữ liệu thời gian thực (Streaming) và viết các kịch bản kiểm thử bảo mật tự động bằng Postman.

3. Khái Quát Định Nghĩa, Kiến Thức Và Công Nghệ Sử Dụng

Để hiện thực hóa dự án, nhóm cần nắm vững các công nghệ lõi sau:

- **RAG (Retrieval-Augmented Generation):** Kiến trúc giúp LLM "đọc" kho tài liệu nội bộ trước khi trả lời, giảm ảo giác (hallucination).
- **Vector Database (Pinecone/ChromaDB):** CSDL chuyên dụng lưu trữ và tìm kiếm văn bản dưới dạng các chuỗi số toán học (vectors/embeddings).
- **LangChain & Hugging Face:** Khung phần mềm (framework) điều phối các tác vụ AI. Nhóm sẽ tích hợp các LLM ngoại tuyến hoặc API từ Hugging Face thay vì phụ thuộc hoàn toàn vào OpenAI.
- **Guardrails (Rào cản bảo mật):** Cơ chế đánh giá chất lượng. Hệ thống sẽ sử dụng thư viện như DeepEval để đo độ tự tin (Answer Relevancy). Nếu điểm $\text{Score} < 0.7$, API sẽ từ chối hiển thị và yêu cầu model tạo lại câu trả lời.
- **Retry/Backoff (FastAPI & Tenacity):** Cơ chế toán học chống sụp đổ hệ thống khi gọi LLM API. Công thức tiêu chuẩn: $\text{Delay} = \text{Base} \times 2^{\text{Attempt}} + \text{Random_Jitter}$. Kết hợp FastAPI để xử lý hàng ngàn request bất đồng bộ.
- **Streamlit:** Thư viện Python giúp Frontend Developer xây dựng giao diện ứng dụng AI nhanh chóng có hỗ trợ web streaming.
- **Docker, Postman, GitHub:** Docker để đóng gói ứng dụng (tránh lỗi "chạy được trên máy tôi nhưng lỗi trên máy bạn"); Postman để bắn request kiểm thử Guardrails; GitHub (GitHub Flow) để quản lý mã nguồn song song.

4. Phân Tích Thị Trường (LinkedIn 2026) Và Tính Thực Tế

Năm 2026, thị trường tuyển dụng không còn mặn mà với các chứng chỉ lý thuyết. Theo báo cáo từ LinkedIn, kỹ năng Kỹ sư AI (AI Engineer) tập trung vào "triển khai", "tối ưu" và "MLOps" là ưu tiên hàng đầu của các doanh nghiệp. Nhà tuyển dụng khao khát những ứng viên có khả năng biến LLM thành công cụ kinh doanh an toàn. Việc bạn sở hữu một dự án có áp dụng **Guardrails**, **FastAPI Backend**, và **Dockerization** chứng minh bạn không chỉ biết "gọi API", mà biết thiết kế hệ thống phần mềm AI có khả năng chịu tải và tự bảo vệ. Đăng tải quá trình này lên LinkedIn kèm README chuẩn doanh nghiệp (chia sẻ về thách thức, cách chia module 4 người) sẽ là "thỏi nam châm" thu hút nhà tuyển dụng ngành IT.[1]

5. Những Bất Lợi Và Bài Toán Chi Phí Tối Ưu (FinOps

hướng tới $0 - $5/tháng)

Với định hướng dự án làm Portfolio, có thể đưa lên Website chạy mượt mà không cần chịu tải hàng chục ngàn Request cùng lúc, nhóm hoàn toàn có thể tối thiểu hóa chi phí xuống mức cực thấp:

- **Chi Phí API và Token LLM (Giảm về $0):** Thay vì dùng OpenAI/Gemini trả phí, nhóm sử dụng **Hugging Face Serverless Inference API**. Nền tảng này cung cấp hạn mức gọi các mô hình mã nguồn mở (như Llama, Mistral) miễn phí cho các nhà phát triển thử nghiệm. Kết hợp với kỹ thuật tìm kiếm lai (Hybrid Search) để nhồi ít văn bản hơn vào prompt, chi phí token được đưa về 0.
- **Chi Phí Lưu Trữ Vector DB (Giảm về $0):** Thay vì thuê Pinecone hay Azure AI Search vốn tốn hàng chục đô la mỗi tháng, nhóm sử dụng **ChromaDB chạy cục bộ (Self-hosted)**. Vector Database sẽ chạy như một dịch vụ nội bộ bên trong Docker Container của bạn, dữ liệu lưu thẳng vào ổ cứng ảo (Docker Volume), không tốn một đồng chi phí duy trì DB trên Cloud.
- **Khó Khăn Hạ Tầng Hosting (Giảm về $3 - $5/tháng):** Để chứng minh năng lực triển khai thực tế (Deploy) thay vì chỉ chạy trên máy tính cá nhân, nhóm có thể thuê các máy chủ ảo (VPS) cấu hình cơ bản từ các nhà cung cấp giá rẻ. Trong năm 2026, các nền tảng như Vultr, Hetzner hoặc các nhà cung cấp nội địa Việt Nam cung cấp VPS Linux với giá dưới $5 (~100.000 VNĐ - 120.000 VNĐ/tháng). Khi kết hợp cấu trúc Docker Compose 3 vùng chứa (FastAPI + Streamlit + ChromaDB), một VPS 2GB-4GB RAM là hoàn toàn đủ sức gánh hệ thống mượt mà để các bạn quay Video Demo hoặc cho nhà tuyển dụng trải nghiệm trực tiếp.

---

6. Lộ Trình Cấp Độ Tháng (Từ 13/04/2026 Đến 31/12/2026)

Tháng / Giai đoạn	**AI 1 (Data & Vector)**	**AI 2 (LangChain & LLM)**	**Web 1 (Backend & Ops)**	**Web 2 (Frontend & QA)**	**Ghi chú (Khó khăn & Lợi ích)**
Tháng 4 (Khởi tạo hệ thống)	Phân tích định dạng file PDF, thiết lập Unstructured.	Nghiên cứu tài liệu Hugging Face, cấu hình API	Khởi tạo repo GitHub chung, viết tệp Docker Compose lõi.	Thiết kế wireframe UI, tạo Postman Workspace cho nhóm.	Khó khăn: Xung đột Git ban đầu. Lợi ích: Hình thành khung làm việc

		Keys.			chuẩn.
Tháng 5 (Xây dựng luồng dữ liệu)	Lập trình pipeline Chunking theo bố cục PDF, làm sạch text.	Viết hàm gọi Hugging Face API cơ bản, cấu hình Prompt.	Dựng server FastAPI rỗng, tạo thư mục kiến trúc dự án.	Setup khung Streamlit, kết nối tới FastAPI rỗng.	Lợi ích: Luồng dữ liệu (pipeline) đã có thể chạy từ đầu đến cuối trên text tĩnh.
Tháng 6 (Tích hợp Vector DB)	Cài đặt ChromaDB, chuyển text thành embeddings và lưu trữ.	Tích hợp thuật toán truy xuất (Retriever) vào LangChain.	Viết API tiếp nhận file upload và đẩy sang dịch vụ Ingestion.	Xây dựng luồng upload file trên UI và hiển thị trạng thái.	Khó: ChromaDB có thể xung đột phiên bản. Lợi ích: Chatbot có "trí nhớ" cơ bản.
Tháng 7 (Logic RAG đa bước)	Tối ưu hóa metadata filtering (lọc vector theo quyền/phân loại).	Xây dựng Agentic RAG: Phân rã câu hỏi, truy xuất lai.	Thiết lập cơ chế Retry/Backoff cho các lệnh gọi LLM API.	Tối ưu hiển thị text phản hồi, chuẩn bị cơ chế Streaming.	Khó: Quản lý chi phí token. Lợi ích: Chất lượng câu trả lời RAG nâng cấp rõ rệt.
Tháng 8 (Bảo mật Guardrails)	Mở rộng pipeline hỗ trợ nhiều định dạng tài liệu hơn.	Viết Middleware DeepEval kiểm tra Confidence Score > 0.7.	Đồng bộ Timeout và Circuit Breaker trên FastAPI.	Viết kịch bản Postman tự động bắn request kiểm tra Guardrails.	Khó: Logic chặn phản hồi khi score < 0.7 gây trễ hệ thống. Lợi ích: Mô hình an toàn.
Tháng 9 (Trải nghiệm	Khắc phục các lỗi gãy	Cấu hình LLM trả về kết quả	Áp dụng Async/Await	Bắt luồng stream trên	Khó: Mất bộ nhớ hội thoại

Streaming)	bảng biểu tài chính khi Chunking.	dạng chunk/stream (Yield).	và StreamingResponse trong FastAPI.	Streamlit, hiển thị text mượt mà.	khi stream. Lợi ích: UI đạt chuẩn tương tác giống ChatGPT.
Tháng 10 (Hoàn thiện tính năng)	Đảm bảo tính toàn vẹn dữ liệu khi khởi động lại Docker.	Thêm tính năng Reranker để chọn lọc top 5 ngữ cảnh tốt nhất.	Viết Unit Test cho Backend, dọn dẹp mã nguồn rác.	Hiển thị trích dẫn tài liệu gốc (Citations) trên màn hình UI.	Lợi ích: Giải quyết triệt để vấn đề ảo giác và minh bạch nguồn dữ liệu.
Tháng 11 (Đóng gói MLOps)	Bàn giao tài liệu cấu trúc Vector cho nhóm Web.	Freeze phiên bản LangChain, đóng gói requirements.txt.	Viết Dockerfile Multi-stage cho Backend và Frontend.	Tích hợp Postman tests vào luồng GitHub Actions (CI/CD).	Khó: Tối ưu dung lượng Docker Image < 500MB. Lợi ích: Ứng dụng chạy độc lập.
Tháng 12 (Portfolio & Launch)	Viết báo cáo đánh giá chất lượng Vector Ingestion.	Tinh chỉnh prompt cuối, viết báo cáo thuật toán RAG.	Đẩy ứng dụng lên VPS chạy live trên Internet.	Hoàn thiện README.md, quay video demo, đăng tải LinkedIn.	Khó: Deploy lên server thực tế bị lỗi cổng. Lợi ích: Hoàn tất dự án xuất sắc, CV mạnh mẽ.

---

7. Lộ Trình Chi Tiết Từng Tuần (Tháng 4/2026)

Phân Tích Mục Tiêu Tháng 4 Và Chuyển Giao Từ Tuần 1 Sang Tuần 2

Mục tiêu cốt lõi của Tháng 4: Lộ trình tháng 4 mang tên "Khởi tạo hệ thống". Nhiệm vụ tối cao của tháng này không phải là làm cho AI trả lời xuất sắc, mà là xây dựng thành công một bộ xương kiến trúc (Microservices) nơi các thành phần FastAPI, Streamlit, và ChromaDB có thể "bắt tay" giao tiếp mượt mà với nhau thông qua Docker.

Nhìn lại Tuần 1 (13/04 - 19/04): Nhóm đã đi một nước cờ "End-to-End" siêu tốc. Lợi ích lớn nhất là đập tan rào cản "làm việc cô lập" – luồng dữ liệu thô đã chạy được từ màn hình Web 2 xuyên qua Backend Web 1, chạm tới DB của AI 1 và lấy phản hồi từ LLM của AI 2. Tuy nhiên, hệ thống này cực kỳ mỏng manh, tựa như một bản nháp: AI chưa có khả năng nhớ lịch sử chat, PDF nhiều trang sẽ bị gãy cấu trúc, và hệ thống sẽ sập ngay nếu Hugging Face API báo lỗi.

Định hướng Tuần 2 (20/04 - 24/04): Tuần 2 được thiết kế lại thành mô hình **5 ngày làm việc cường độ cao** (Nghỉ T7, CN). Tuần này đóng vai trò là "Tuần Lễ Gia Cố" (Hardening Week). Mục tiêu của Tuần 2 là xử lý dứt điểm các "nợ kỹ thuật" của Tuần 1. Bằng cách thiết lập bộ nhớ hội thoại (Session State), minh bạch nguồn trích dẫn (Citations) và xử lý ngoại lệ (Error Handling), Tuần 2 tạo ra một ranh giới chốt chặn (Baseline) vững chắc. Nó giải quyết sự mong manh của Tuần 1, tạo tiền đề an toàn tuyệt đối để Tuần 3 và 4 có thể tự tin tích hợp các thuật toán truy xuất phức tạp mà không sợ làm sập hệ thống web.

Lộ Trình 7 Ngày Đầu Tiên Bứt Tốc (Tuần 1: 13/04 - 19/04/2026)

Ngày	**AI 1 (Data & Vector)**	**AI 2 (LangChain & LLM)**	**Web 1 (Backend & Ops)**	**Web 2 (Frontend & QA)**	**Ghi chú (Khó khăn & Lợi ích)**
Ngày 1 (13/04)	Cài Python 3.10. Tích hợp ngay thư viện PyMuPDF và Unstructured cho phân rã PDF.	Tạo tài khoản Hugging Face, sinh Token. Viết script Python gọi API Hugging Face Serverless.	Thiết lập repo GitHub áp dụng chiến lược GitHub Flow (1 nhánh Main duy nhất). Setup Docker.	Tạo Postman Workspace chung. Thiết kế Wireframe UI trên Figma cho khung chat và nút upload.	**Khó khăn:** Đồng bộ môi trường hệ điều hành khác nhau. **Lợi ích:** Nền tảng công cụ sẵn sàng ngay ngày 1.
Ngày 2 (14/04)	Viết script đọc 1 file PDF mẫu, bóc tách text thô. Xóa ký tự lạ, làm sạch (Regex).	Cài LangChain. Tạo PromptTemplate cơ bản đóng vai trò là "Trợ lý phân tích doanh nghiệp".	Viết docker-compose.yml nền tảng để chạy ChromaDB nội bộ và tạo Docker Network Bridge.	Khởi tạo khung dự án Streamlit (app.py), code giao diện sidebar tĩnh (chưa có chức năng).	**Khó khăn:** Cấu hình mạng nội bộ Docker. **Lợi ích:** Các khối code độc lập (Microservices) bắt đầu hình thành.

Ngày 3 (15/04)	Áp dụng Layout-aware chunking, code thuật toán chia file PDF theo từng đoạn/trang hợp lý.	Bọc logic gọi LLM vào 1 class Python (OOP). Kiểm thử việc tự động chèn ngữ cảnh vào Prompt.	Khởi tạo thư mục chuẩn của FastAPI. Viết main.py với endpoint kiểm tra sức khỏe /health.	Xây dựng cửa sổ hiển thị hội thoại chính trên Streamlit. Tích hợp CSS/Markdown cơ bản.	**Khó khăn:** Chunking làm gãy cấu trúc các bảng biểu. **Lợi ích:** API Backend và Frontend UI đã có khung rỗng giao tiếp được.
Ngày 4 (16/04)	Thiết lập thư viện Embedding, mã hóa thử các khối text vừa bóc tách thành vector cục bộ.	Tích hợp class LLM vào chuỗi LangChain LLMChain. Test độ trễ phản hồi của Hugging Face.	Dùng Pydantic định nghĩa Schema JSON (Input/Output). Viết API /ask nhận JSON câu hỏi.	Dùng thư viện requests nối Streamlit với API /health của Web 1. Bắt sự kiện nhấn nút Gửi.	**Khó khăn:** Schema JSON giữa Front và Back không khớp nhau. **Lợi ích:** Frontend và Backend bắt đầu bắt tay (handshake).
Ngày 5 (17/04)	Khởi tạo ChromaDB client. Bơm các Vector vừa tạo vào CSDL nội bộ.	Viết code ghép RAG giả lập: Nhận câu hỏi -> Nhét đoạn văn cố định -> Hỏi LLM -> Trả kết quả.	Viết API /upload tiếp nhận file PDF. Đóng gói mã nguồn FastAPI thành file Dockerfile.	Cấu hình UI để hiển thị chuỗi JSON rỗng từ Backend. Viết luồng xử lý lỗi khi API báo lỗi 500.	**Khó khăn:** Quản lý Volume của Docker để Database không mất dữ liệu. **Lợi ích:** Luồng RAG sơ khai được định hình.
Ngày 6 (18/04)	Đóng gói mã Ingestion thành 1 file Python chạy độc lập. Viết script xử lý	Kết nối công cụ truy xuất của ChromaDB vào chuỗi LangChain.	Nối API /upload với module xử lý file của AI 1. Đóng gói Streamlit	Viết các bộ kiểm thử tự động trên Postman để bắn 50 request vào	**Khó khăn:** Ghép nối code của đội AI vào hệ thống Backend Web. **Lợi ích:** Dữ liệu

	luồng (Batch) 10 file.	Chạy thử hỏi đáp thực tế.	thành Dockerfile.	/ask test độ tải.	chạy xuyên suốt từ Upload -> AI -> UI.
Ngày 7 (19/04)	Rà soát lỗi bóc tách siêu dữ liệu (Metadata). Bắn Pull Request mã nguồn lên GitHub.	Kiểm tra nhanh hiện tượng ảo giác của RAG v1. Bắn Pull Request lên GitHub.	Lên cấu hình docker-compose up -d kích hoạt cùng lúc 3 containers (API, Web, DB).	Chạy bộ test Postman. Review code trên GitHub, bấm hợp nhất (Merge) PR vào nhánh chính.	**Khó khăn:** Gỡ lỗi (Debug) môi trường liên vùng chứa. **Lợi ích:** Hoàn tất nguyên mẫu Chatbot v1 cực tốc độ.

Lộ Trình 5 Ngày Tinh Chỉnh Nền Tảng (Tuần 2: 20/04 - 24/04/2026)

Ngày	AI 1 (Data & Vector)	AI 2 (LangChain & LLM)	Web 1 (Backend & Ops)	Web 2 (Frontend & QA)	Mục Tiêu, Lý Do & Điểm Kết Nối
Thứ 2 (20/04)	Áp dụng Chunking Overlap (khoảng lặp) để giữ tính liên kết ngữ nghĩa giữa các đoạn cắt PDF.[2]	Cấu hình lại PromptTemplate để có thể nhận thêm mảng lịch sử hội thoại (Chat History).	Tích hợp Pydantic để rà soát chặt chẽ và từ chối các chuỗi JSON đầu vào sai định dạng.	Kích hoạt st.session_state để UI lưu giữ lịch sử chat, không bị xóa sạch mỗi lần tải lại trang.	**Mục tiêu:** Ổn định bộ nhớ. **Lý do:** Khắc phục tình trạng "mất trí nhớ" của tuần 1. Đây là điểm nối bắt buộc để tiến tới tính năng trò chuyện đa vòng (Multi-turn) vào cuối

					tháng.
Thứ 3 (21/04)	Code script trích xuất siêu dữ liệu cơ bản (Tên file, Số trang) để đính kèm vào Vector.[3]	Viết lớp Memory Buffer bọc ngoài LLM để tự động nhồi lịch sử chat vào Hugging Face.[5]	Cập nhật API /ask để nhận mảng danh sách (Array) các câu chat cũ thay vì 1 câu string đơn lẻ.	Viết hàm chuyển đổi dữ liệu Session State thành mảng JSON chuẩn xác để đẩy xuống API Backend.	**Mục tiêu:** Đồng bộ logic hội thoại. **Lý do:** Khớp nối logic lưu trữ cục bộ (Frontend) với logic xử lý hệ thống (Backend). Tránh xung đột luồng dữ liệu trước khi ghép các module lớn.
Thứ 4 (22/04)	Thiết lập cấu trúc đẩy siêu dữ liệu (Metadata) vào ChromaDB song song với Vector Embeddings.[6]	Chỉnh sửa module Truy xuất (Retriever) để mô hình trả về cả Text lẫn Metadata của tài liệu nguồn.[5]	Bổ sung trường citations (Nguồn trích dẫn) vào Schema trả về (Response Schema) của FastAPI.	Thiết kế UI dạng hộp thoại thả xuống (Expander) ở dưới mỗi câu trả lời để hiển thị Nguồn tham khảo.[7]	**Mục tiêu:** Minh bạch nguồn gốc. **Lý do:** Trọng tâm của tháng 4 là hình thành cấu trúc ứng dụng. Việc chuẩn bị luồng hiển thị Citations là bước đệm trực tiếp để triển khai Guardrails (bảo mật) ở các tháng sau.

Thứ 5 (23/04)	Viết logic dọn dẹp CSDL (CRUD): Tự xóa Vector cũ nếu user upload lại file trùng tên.	Đánh giá giới hạn Rate Limit của HF API, bắt lỗi Time-out nếu mô hình bị treo và phản hồi chậm.	Bọc Try/Catch toàn diện cho API, cấu hình trả về mã lỗi chuẩn (HTTP 429, 500) giúp bảo vệ server.	Viết Postman script giả lập ép lỗi Backend, lập trình cho Streamlit hiển thị cảnh báo (Toast error) thân thiện.	**Mục tiêu:** Xử lý ngoại lệ. **Lý do:** Tuần 1 hệ thống chỉ chạy đường thẳng (happy path) và rất dễ sập. Bắt lỗi sớm giúp nền tảng vững chắc, sẵn sàng cho kiểm thử UAT cuối tháng.
Thứ 6 (24/04)	Đóng gói bản cập nhật Data Ingestion vào Dockerfile, dọn dẹp các thư viện rác trong môi trường.	Đóng băng (Freeze) các phiên bản thư viện LangChain trong requirements.txt để chống lỗi tương thích.	Chạy lệnh docker-compose up --build để khởi động lại 3 container, test kiểm chứng giao tiếp mạng nội bộ.	Chạy toàn bộ Test Runner trên Postman. Review code chéo và hợp nhất (Merge) PR vào nhánh Main trên GitHub.	**Mục tiêu:** Đóng gói & Hợp nhất. **Lý do:** Chốt hạ kết quả Tuần 2. Đảm bảo đạt được 50% mục tiêu của Tháng 4 là sở hữu một hệ thống RAG độc lập, chạy tự động và an toàn trên Docker.

Các Tuần Tiếp Theo (Từ Tuần 3, Tháng 4/2026 - Tháng 12/2026)

(Lưu ý: Sau khi trải qua hai tuần đầu tiên để dựng khung xương và gia cố tính ổn định, các tuần tiếp theo sẽ chuyển trọng tâm sang sự tinh chỉnh độ chính xác (Reranker), bảo mật MLOps (Guardrails) và trải nghiệm người dùng (Streaming Responses). Lộ trình các tháng tiếp theo

được giữ nguyên như bảng tổng quan phía trên).

8. Lời Kết Và Kế Hoạch Chinh Phục Nhà Tuyển Dụng

Mô hình làm việc bứt tốc ở Tuần 1 và gia cố tập trung ở Tuần 2 đóng vai trò định hình kỷ luật MLOps cho toàn bộ vòng đời phát triển dự án. Cả 4 thành viên sẽ không làm việc rời rạc, mà tích hợp công việc vào nhau qua các bản thiết kế API JSON, Docker Network, và GitHub Branching. Bằng cách tận dụng Hugging Face Serverless, ChromaDB nội bộ và VPS giá cực rẻ (< 150K/tháng), nhóm không chỉ hiện thực hóa được hệ thống RAG doanh nghiệp một cách tối ưu ngân sách, mà còn nắm trong tay câu chuyện chứng minh năng lực tài chính đám mây (FinOps) – một điểm cộng cực lớn trong mắt các nhà tuyển dụng công nghệ IT năm 2026.

Works cited

1. Create a GitHub README Template for Projects ... - SocialPrachar, accessed April 11, 2026, https://socialprachar.com/blog/create-a-github-readme-template-for-projects-today
2. Mastering Git Branching Strategies: Finding the Right Fit for Your Team - DEV Community, accessed April 11, 2026, https://dev.to/trinitezinsou/mastering-git-branching-strategies-finding-the-right-fit-for-your-team-4p2n
3. Agentic RAG in 2026: The UK/EU enterprise guide to grounded ..., accessed April 11, 2026, https://datanucleus.dev/rag-and-agentic-ai/agentic-rag-enterprise-guide-2026
4. Building Production-Grade AI Guardrails: A Deep Technical ..., accessed April 11, 2026, https://medium.com/@_Ankit_Malviya/building-production-grade-ai-guardrails-a-deep-technical-implementation-guide-7457db3ea510
5. LLM RAG Tutorial: Examples and Best Practices | LaunchDarkly, accessed April 11, 2026, https://launchdarkly.com/blog/llm-rag-tutorial/
6. 10 Effective Steps To Building RAG Applications - Techment, accessed April 11, 2026, https://www.techment.com/blogs/building-rag-applications-enterprise-guide/
7. RAG Chatbot with Conversational Memory using LangChain | by ..., accessed April 11, 2026, https://tuanatran.medium.com/rag-chatbot-with-conversational-memory-using-langchain-24b5886de244